

PROJECT REPORT

SQL Data Analysis

DecodeLabs Data Analytics Internship
Industrial Training Kit | Batch 2026

Project Title	SQL Data Analysis
Organization	DecodeLabs
Program	Data Analytics Internship – Industrial Training Kit
Batch	2026
Tool Used	SQL (SQLite) + Python/Pandas
Date	11 September 2026

1. Project Objectives

This report documents the SQL Data Analysis performed on the cleaned e-commerce order dataset. The objective was to extract precise, actionable insights using structured queries — filtering, grouping, aggregating, and sorting data through pure relational logic.

SQL is a declarative language. Instead of writing step-by-step procedural instructions, the analyst defines the desired output and lets the database engine determine the most efficient execution path. Mastering SELECT, WHERE, GROUP BY, ORDER BY, and aggregations is the foundation of professional data extraction.

2. Project Objectives

- Write clean and correct SELECT queries.
- Filter rows using WHERE (equality, comparison, multiple conditions).
- Sort results with ORDER BY.
- Group data with GROUP BY and apply aggregations (COUNT, SUM, AVG).
- Use HAVING to filter aggregated results and produce business-ready summaries.

3. Dataset & Environment

The cleaned order dataset (1,200 rows) was loaded into a SQLite database table named orders. This approach keeps the solution fully portable and reproducible without requiring an external database server.

Attribute	Value
Database	SQLite (orders.db)
Table	orders
Total Rows	1,200
Key Columns	OrderID, Product, Quantity, TotalPrice, OrderStatus, PaymentMethod, ReferralSource

4. Methodology

Twelve analytical queries were designed to cover every required SQL skill. Each query was executed against the orders table, results were inspected, and output tables were exported for documentation.

4.1 Filtering (WHERE)

WHERE clauses isolated high-value orders (TotalPrice > 2000), cancelled orders, and multi-condition subsets (e.g., Cancelled AND Quantity ≥ 4). This demonstrates row-level filtering before any aggregation.

4.2 Grouping & Aggregation

GROUP BY combined with COUNT, SUM, and AVG produced category-level insights: revenue by product, order status distribution, payment method performance, and referral source effectiveness.

4.3 Post-Aggregation Filtering (HAVING)

HAVING was used to keep only products whose average order value exceeded \$1,000. This shows understanding of the logical execution order (HAVING runs after GROUP BY).

5. Key Results

Metric	Value	Source Query
Total Orders	1,200	Business Summary
Total Revenue	\$1,264,761.96	Business Summary
Avg Order Value	\$1,053.97	Business Summary
Cancelled Orders	250 (20.8%)	Status Distribution
Returned Orders	247 (20.6%)	Status Distribution
Top Revenue Product	Chair / Printer (~\$195.6k)	Revenue by Product

6. Business Insights

- Cancellation and return rates remain high (~41% combined). SQL makes this visible instantly via GROUP BY on OrderStatus.
- Revenue is relatively balanced across the seven product categories; no single product dominates.
- High-value orders (TotalPrice > 2000) can be isolated in one WHERE clause for VIP or fraud review.
- HAVING lets us focus only on product segments that meet a minimum average order value threshold — useful for prioritization.

7. Deliverables

- orders.db – SQLite database containing the orders table
- sql_analysis.py – Script that executes all 12 analytical queries
- sql_outputs/ – Excel result tables for every query
- Project_Report.pdf – This formal project report
- README.md – Project documentation for GitHub

8. Conclusion

This project successfully demonstrated the core SQL skills required for professional data extraction: precise filtering, categorical grouping, aggregations, and post-aggregation logic. By thinking in terms of the database execution order (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY), the queries remain both correct and efficient.

SQL is the language of truth extraction. This milestone proves the ability to turn a raw table into clear, decision-ready answers.